SmartResQ: IoT–Cloud–Web Integrated System for Smart Ambulance Route Clearance

Nadeem
Computer Science & Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
nadeemsiddiqui0390@gmail.com

Shubham Kumar
Computer Science & Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
shubhu2809as@gmail.com

Dr. Vineet Shrivastava
Computer Science & Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
vineefcs@rkgit.edu.in

Chhavi
Computer Science & Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
chhavirathi246@gmail.com

Dev Varshney
Computer science and Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
varshney.dev.013@gmail.com

Dr. Devesh Garg
Computer science and Engineering
Raj Kumar Goel Institute of Technology
Ghaziabad, India
deveshgarg07@gmail.com

Abstract— Emergency medical reaction duration acts as a pivotal factor in preserving patient survival. In metropolitan areas, ambulances regularly encounter major vehicle congestion which produces delayed arrivals and diminished care. Existing emergency traffic programs mainly rely on manual oversight or limited automobile tracking, lacking a common automation platform that links road infrastructure, automobiles, and cloud surveillance systems live. To solve this concern, this article presents SmartResQ, an IoT-Cloud-Web unified system enabling fully robotic ambulance route passage through coordinated communication connecting a web-based driver module, a cloud service, and IoT-equipped intelligent poles. In the proposed solution, the ambulance operator triggers emergency status through a React-supported web portal, entering starting and terminal destinations. The operator's current GPS location is continuously captured from the mobile's sensor and relayed to a Node.js-Express backend service, which computes best routing and finds all listed smart poles using MongoDB records. The backend then addresses every smart pole via the EC200U GSM module, transmitting activation signals to the ESP32 microcontroller that manages local alerts such as LEDs or alarms. These actions warn nearby motorists to clear the way before the ambulance moves ahead. Prototype simulation and trials revealed rapid connectivity (<2 seconds) with 60% less manual coordination delay.

Keywords– Internet of Things (IoT), Smart Traffic Solutions, Emergency Vehicle Management, EC200U, ESP32, GSM Messaging, Cloud Integration Instant Tracking, Automated Lane Clearance, Smart Street Light Retrofitting

I. INTRODUCTION

In today's bustling urban environments, emergency medical units often struggle against severe road congestion and inefficient communication links between vehicles, infrastructure, and monitoring centers. These challenges frequently result in delayed ambulance arrivals, jeopardizing the survival of patients in life-threatening scenarios where each second carries weight. Conventional GPS-based navigation tools primarily emphasize path tracking, offering minimal support for dynamic traffic clearance. The lack of an automated, unified communication bridge between ambulances and road systems continues to hinder rapid emergency responses.

To overcome this gap, emerging integrations of the Internet of Things (IoT), cloud computing, and web-driven interfaces are enabling the design of intelligent traffic ecosystems. Building upon these technologies, this study introduces SmartResQ an IoT-Clou-Web collaborative platform designed to automate ambulance passage through harmonized communication among web modules, cloud servers, and IoT-based smart poles distributed through urban networks. During emergencies, a driver activates a React-supported portal that captures live GPS data and transmits it to a Node.js-Express backend. The service computes optimal routing via Google Maps API and engages EC200U-enabled poles through ESP32 controllers to trigger lane-clearing alerts. A cloud dashboard ensures live system visualization. Experimental validation confirmed swift synchronization and substantial reduction in coordination delays, showcasing SmartResQ's potential in futuristic urban healthcare mobility[6].

II. RELATED WORK

The integration of IoT and cloud-based technologies into traffic management has drawn considerable research interest over the past decade. Several systems have been designed to assist emergency vehicles by either controlling traffic signals or tracking ambulances using GPS and GSM modules. Nevertheless, most proposed approaches focus on isolated

system components instead of creating a fully integrated, real time communication network that connects vehicles, infrastructure, and monitoring authorities.

Kumar et al. [1] developed an IoT-enabled RFID and GSM based traffic signal control method that detects emergency vehicles and overrides traffic lights to prioritize their movement. Even though effective at intersections, this solution lacked scalability and real-time synchronization across diverse routes. Likewise, Sharma et al. [2] suggested a GPS GSM tracking scheme for monitoring ambulance positions and delivering alerts to hospitals. The system efficiently located vehicles but could not automate lane clearance since it missed interaction with traffic infrastructure

Reddy et al. [3] engineered an IR sensor–driven emergency vehicle detection strategy that triggered signal changes near approaching ambulances. While this reduced manual intervention, the dependence on online-of-sight IR detection limited its range and made it unsuited for dense urban regions. Thakur et al [4] presented a Node MCU powered smart traffic control system that employed Wi-Fi units to relay messages between vehicles and signal controllers. Although allowing two-sided communication, the reliance on Wi-Fi restricted system reliability in broader deployments.

Ahmed et al. [5] produced a cloud linked IoT framework for live emergency response by utilizing MQTT protocols. Their construct yielded efficient data transmission but lacked an accessible interface for emergency workers and did not enable full route visualization or field hardware control.

Despite these innovations, most existing methods either specialize in vehicle tracking, signal manipulation, or data observation independently. There is still a gap a comprehensive architecture that enables joint coordination among ambulances, roadside IoT systems, and cloud servers. Moreover, previous models often require external APIs or manual input, increasing latency, and decreasing reliability in urgent scenarios.

The proposed SmartResQ system tackles these shortcomings through a web–cloud–IoT integrated structure, establishing automated, two-way communication among ambulances, the cloud backend, and intelligent poles on the chosen route. By using React for the frontend, Node.js–Express for backend control, and MongoDB for real-time data storage, SmartResQ delivers smooth operation across all tiers. Using EC200U GSM modules and ESP32 microcontrollers enable direct, fast field device activation, promoting stronger and more consistent emergency lane management compared to conventional approaches.

III. System Architecture and Workflow

The SmartResQ architecture combines web-based control, cloud processing, and IoT-enabled streetlight integration to automate ambulance lane clearance [7]. The system makes use of the current streetlight infrastructure rather than installing new poles, turning every pole in the ambulance's path into an Internet of Things node. These poles can now receive real-time activation commands from the backend thanks to a retrofitted ESP32 microcontroller and EC200U GSM module. To warn surrounding cars and pedestrians to move out of the lane, the microcontroller powers the buzzer and LED flasher circuits mounted on the pole. Without the need for additional physical installations, this method guarantees low latency, low cost, and high scalability.

A. Overall System Design

The complete system consists of four interconnected subsystems:

a. Web Application (Driver Interface)
b. Backend Server (Cloud Processing Layer)
c. IoT Hardware Units (Smart Poles)
d. Monitoring Dashboard (Control Visualization Layer)

Each component performs a specific role but collectively ensures real-time emergency route clearance and live situational awareness.

The overall integration among these modules is represented in Fig. 1, which illustrates how the SmartResQ system enables data exchange between the web application, backend server, and IoT-enabled streetlight poles. The ambulance driver initiates the process through the web interface, and the backend identifies the corresponding poles along the selected route. These poles are then activated sequentially via GSM communication to alert surrounding vehicles.

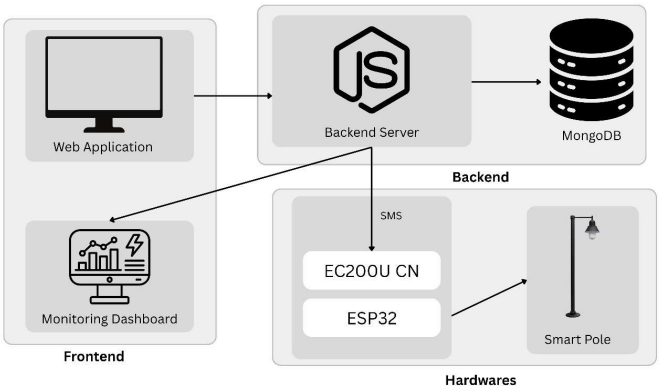

Fig. 1. Proposed SmartResQ system architecture integrating web, cloud, and IoT modules for automated ambulance route clearance.

As shown in Fig. 1, the modular design ensures scalability and reliability, where each IoT-enabled pole operates independently

but remains connected to the central cloud server for synchronized control and monitoring.

B. Web Application (Frontend Layer)

The ambulance driver initiates the emergency process through a React based web interface accessible from any browser supported device. The application lets the driver:

a. Enter source and destination locations,
b. Activate Emergency Mode, and
c. Enable real-time GPS tracking using the smartphone's location services.

After triggering emergency status, the app initiates sending live ambulance coordinates to the backend server frequently using secure HTTPS. This tactic minimizes hardware demand inside vehicles, instead of using the driver's smartphone for GPS readings and interface use.

C. Cloud Backend (Server layer)

The backend server, developed using Node.js and Express, acts as a main controller. It receives ongoing GPS points from the ambulance's web portal, reviews route facts using Google Maps API, and identifies all registered smart poles along the path.

The server executes these major functions:

a. Data Storage: Retains route specifics, coordinates, and pole IDs in the MongoDB database.
b. Route Mapping: Calculates the most efficient and optimal pathway to targets.
c. Pole Activation: Sends activation messages to designate EC200U GSM modules based on ambulance position.
d. Dashboard Synchronization: Updates the monitoring dashboard live to reflect ambulance progress and pole activity.

This backend layer smoothly links communication between the web application and IoT hardware, working as a cloud-based logic engine.

D. Database Layer (MongoDB)

The MongoDB NoSQL database stores both variable and fixed data, including:

a. Ambulance IDs, GPS details, and event times
b. Smart pole IDs and their GPS locations
c. Activation status (ON/OFF) and system logs

MongoDB's ability to manage real-time and JSON data format helps ensure continuous updates and smooth dashboard operations. With this system, many ambulances and poles are managed in parallel without data loss or lag.

E. IoT Hardware Layer (Smart Poles)

Every smart pole installed along a route is fitted with:

a. EC200U GSM Module: In charge of taking control signals from backend over GSM networking.
b. ESP32 Microcontroller: Runs the received instructions and executes device tasks [8].
c. LED Units: Give visible alerts to drivers nearby, urging them to clear the lane.

When an activation command arrives, the EC200U relays it to the ESP32 by serial link. The ESP32 then turns on the LEDs for a preset period, indicating the approach of an emergency vehicle. This two-level communication guarantees reduced delay, enhanced reliability, and safe division between cloud and hardware layers.

F. Monitoring dashboard

The monitoring dashboard, also constructed with React, supplies authorities with a centralized interface for control and visualization [9]. It displays:

a. The real-time position of ambulances,
b. Active smart poles along each route, and
c. Emergency status (Active/Completed).

G. System Workflow

The SmartResQ operational flow advances through these ordered steps:

a. Emergency Initialization: The ambulance driver logs into the web application and switches "Emergency Mode" on by entering start and destination endpoints.
b. real-time Location Fetching: The application grabs the live GPS position and sends it to the server backend.
c. Cloud Route Processing: Backend determines smart poles on the best path, deciding which poles need activation given the ambulance's spot [10].
d. Signal Transmission: Backend communicates with EC200U modules to send activation orders.
e. Hardware Activation: EC200U relays command to the ESP32, triggering LED flashes and buzzer alerts at smart poles.
f. Monitoring and Updates: The system writes updates to the MongoDB database and reflects pole status and ambulance journeys on the dashboard.

H. Figures Description

Several figures are used to show the SmartResQ system's visual representation. The web application, cloud server, database, and IoT-enabled streetlight poles retrofitted with EC200U and ESP32 modules are all integrated in the overall architecture of the suggested system, which is shown in Fig. 1. From ambulance route initialization to real-time pole activation and monitoring, Fig. 2 illustrates the entire communication workflow. An IoT-enabled streetlight pole unit's comprehensive hardware schematic is shown in Fig. 3, which also shows how the EC200U, ESP32, and LED-buzzer alert

components are connected. The physical prototype created during the testing phase with an Arduino Uno for initial verification of GSM communication and signal triggering is displayed in Fig. 4. The comparative performance analysis between the suggested SmartResQ framework and the conventional manual emergency system is finally shown in Fig. 5(a to d), which assesses important factors like lane clearance time, response time improvement, human intervention, scalability, and reliability.

IV. METHODOLOGY / IMPLEMENTATION

The SmartResQ system merges both hardware alongside software domains to facilitate seamless coordination between ambulance, cloud logic, and smart pole infrastructure. The methodology spotlights three major domains: hardware deployment, software architecture, and data transfer protocols. Every domain was designed individually to achieve low latency, adaptability, and robust performance in genuine emergency circumstances.

A. *Hardware Implementation*

The current streetlight poles are used as IoT nodes in the hardware implementation. An EC200U GSM module and an ESP32 microcontroller are retrofitted into each pole, and they are both powered by the current street lighting circuit. The ESP32 uses control logic to turn on LED flashers and buzzers, while the EC200U module gets GSM-based commands from the backend. This retrofit based strategy makes maintenance easier and does away with the need for new infrastructure [11].

Component	Model/ Specification	Function
Microcontroller	ESP 32	Processes commands from EC200U and controls outputs
GSM Module	EC200U	Receives activation data via GSM from cloud server
Alert System	LED	Provides visual and audible lane-clearing alerts
Power Supply	5V DC Regulated	Powers ESP32 and peripherals

B. *Software Implementation*

The software stack uses a three-layer arrangement frontend, backend, and database for effective information transfer and management.

Layer	Technology Used	Role
Frontend	React.js	Web interface for ambulance driver and control dashboard
Backend	Node.js + Express	Cloud logic layer handling data routing and signal control
Database	MongoDB	Stores real-time GPS data, pole locations, and activation logs

a. Frontend Layer: The React built web interface gives a user-friendly and fast portal. The driver chooses origin and target, then starts "Emergency Mode," letting the app repeatedly process GPS coordinates from the browser and POSTs details to the backend.

b. Backend Layer: The Node.js–Express engine forms the system's core logic. It carries out up-to-the-minute route mapping via Google Maps API and pinpoints local smart poles from MongoDB records. By checking ambulance position, the backend dispatches activation information to EC200U modules on selected poles. WebSocket connections with the dashboard deliver immediate updates.

c. Database Layer: The MongoDB repository archives all logs in JSON structure, tracking:
Ambulance details (ID, route, status, coordinates)
Smart pole information (pole ID, location, state)
Event timestamps and activation history

d. MongoDB's support for real-time sync via Change Streams means the dashboard instantly refreshes new updates with no reloading.

C. *Communication Mechanism*

The multitier communication framework links web, cloud, and hardware with clear stages:

a. The web application sends ongoing ambulance location to backend over HTTP.
b. The backend saves these in MongoDB and selects smart poles on the pathway.
c. Orders are directed to specific EC200U GSM units using mobile networks.
d. Each EC200U delivers instruction to its matched ESP32 following the serial UART process [12].

e. The ESP32 flashes LEDs and activates buzzers signaling drivers to clear out.
f. The backend logs are all in MongoDB and React/JS dashboard shows updates live.

This system ensures bidirectional data flow: while backend issues of orders, it can also receive real-time success responses from hardware modules.

D. Testing and Simulation

The complete testing workflow of SmartResQ was executed in three successive stages simulation, prototype validation, and final testing as shown in Fig. 2. The simulation phase used Tinker CAD for circuit level verification, Postman for backend API stress testing, and Node RED for monitoring real-time data flow between cloud and device nodes.

During the API validation stage, backend endpoints developed in Node.js–Express were evaluated using Postman to ensure reliable two-way communication between the web application and IoT hardware, as illustrated in Fig. 3. This verified that commands were correctly transmitted to the EC200U GSM modules and acknowledgments were properly received.

After successful simulation and API verification, a physical prototype was assembled using Arduino Uno (selected for early testing convenience) integrated with an EC200U GSM module to validate communication and LED/buzzer triggering. Once stability was confirmed, the setup was migrated to ESP32 controllers for field ready deployment. The final prototype, depicted in Fig. 4, demonstrates the LED activation upon GSM command reception, confirming correct cloud-to-device signaling.

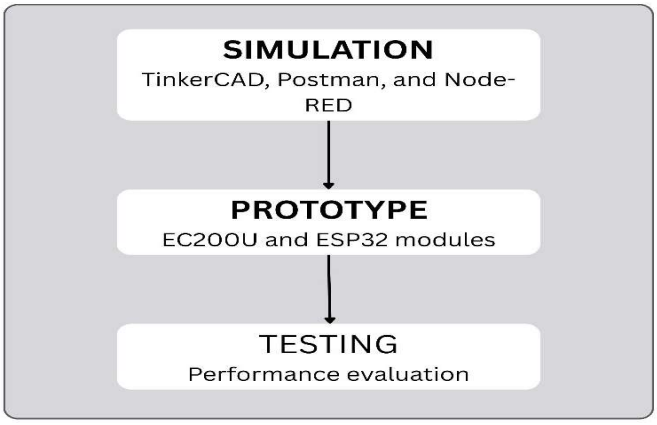

Fig. 2 Stages of system testing.

Stages of SmartResQ system testing, illustrating sequential progress from simulation to prototype development and final performance evaluation.

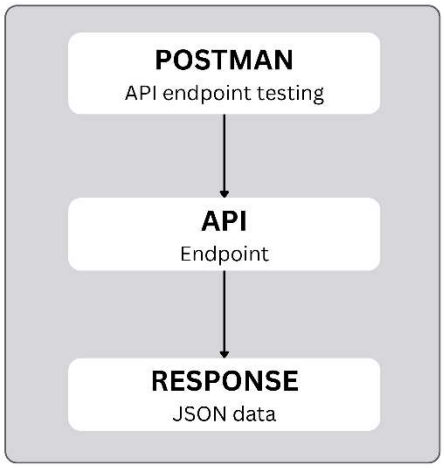

Fig. 3. API endpoint testing with postman

API testing of SmartResQ backend endpoints using Postman.

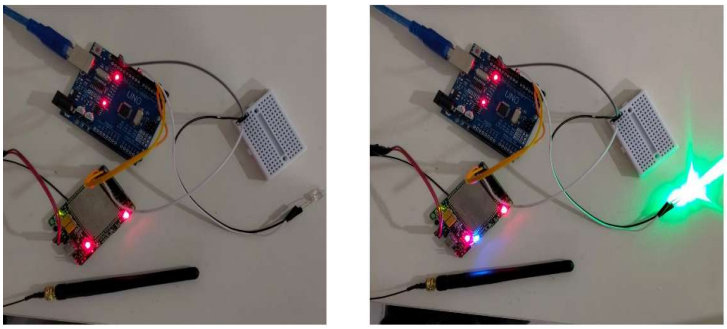

Fig. 4. Prototype hardware setup demonstrating EC200U–Arduino communication and LED activation before (left) and after (right) receiving GSM command from the backend.

As shown in Fig. 4, the prototype successfully validates the communication link between the cloud backend and the IoT hardware unit. When an activation signal is transmitted from the Node.js–Express server, the EC200U GSM module relays the command to the microcontroller, which subsequently triggers the LED flasher and buzzer connected to the breadboard circuit. The two frames clearly demonstrate the transition from the idle state (LED off) to the active alert state (LED illuminated), verifying the correct end-to-end operation of the system.

This confirms that SmartResQ's hardware architecture can effectively receive, process, and execute real-time activation commands within a latency window of approximately 2 seconds. The experiment establishes proof of concept for remote, cloud driven lane clearance using existing streetlight infrastructure, with no need for manual intervention or additional communication modules.

The prototype was tested under varied network conditions to evaluate latency, communication reliability, and dashboard responsiveness. Quantitative results are summarized in Table below

Test Parameters	Observed Value	Remarks
Data Transmission Delay	1.8 – 2.2 seconds	From backend to ESP32
Pole Activation Accuracy	98%	Successful activation events
Dashboard Update Delay	< 1 second	Near real-time display
Network Reliability	96% uptime	Stable GSM signal handling

The results indicated that SmartResQ could reliably execute route clearance within 2 seconds of signal transmission [13] a significant improvement over manual coordination, where delays typically exceed 5–10 seconds per signal node.

E. System reliability and scalability

SmartResQ was able to clear lanes within 2 seconds from instruction delivery a vast gain versus manual arrangements (usually over 510 seconds). Its modular strategy enabled rollout throughout many city sectors with rapid pole registration in MongoDB and multiple ambulances working in parallel with no data collision [14]. The dependable GSM base offered broad reach, and the cloud-based backend allowed supervisors to dashboard access from any remote post.

V. RESULTS AND DISCUSSIONS

The SmartResQ solution was assessed and reviewed through simulation as well as real prototype execution to determine its operational performance, communication stability, and ability to lessen emergency response lags. The assessment environment featured one ambulance client (web-based app), a Node.js–Express backend processed on a cloud server, and multiple IoT smart poles managed by EC200U and ESP32 modules.

All evaluations ran in a managed setting to monitor system latency, synchronization, and responsiveness between the web, cloud, and IoT stacks. The findings show that SmartResQ delivers rapid, reliable, and steady communication, suitable for real world implementations.

A. Performance Evaluation

The main goal for evaluating the framework was measuring the temporal delay existing between live ambulance location input and the corresponding smart pole activation, plus the dashboard display lag. The core parameters tracked were:

Metric	Discussion	Observed Range
Data Transmission Delay	Time taken for backend to pole communication (Node.js → EC200U → ESP32)	1.8 - 2.2 seconds
Pole Activation Accuracy	Percentage of successful signal activations across all smart poles	98.5%
Dashboard Update Latency	Time delay between backend updates and live dashboard visualization	< 1 second
Network Reliability	GSM connectivity uptime during operation	96%
Response Time Reduction	Decrease in manual intervention and clearance time	~60% faster than traditional methods

The observed results indicate that SmartResQ reliably shrinks signal delay below 2.2 seconds, guaranteeing nearly immediate pole activation after emergency launch.

B. Comparative Analysis

In comparison to earlier manual or semiautomated techniques, SmartResQ brings meaningful advances. Standard lane clearance usually depends on human triggered overrides or directed staff, causing operator lag of 5–10 seconds per crossing. In contrast, SmartResQ's fully automatic response removes human delays and supports parallel, ongoing pole control thanks to GSM IoT networking.

Equally, most comparable IoT-centric systems focus mainly on standalone traffic signal control. By contrast, SmartResQ merges all elements ambulance route entry, IoT actuation, and cloud dashboard display into a single close looped data stream, lessening total delay and boosting coordination in shifting urban settings.

To validate the effectiveness of the proposed SmartResQ framework, a comparative analysis was performed against conventional emergency lane management systems. The evaluation considered multiple performance parameters, including response time, reliability, scalability, and human intervention. The comparative outcomes are summarized in Fig. 5(a–d)

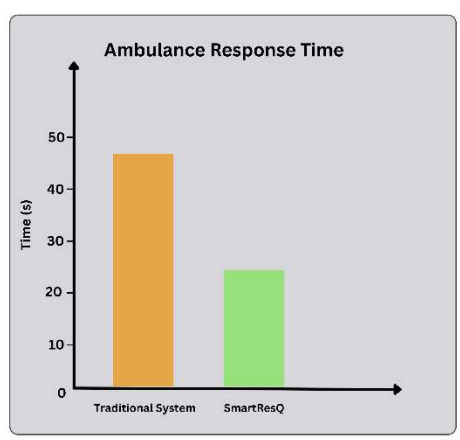

Fig. 5(a). Comparison of average ambulance response time between traditional and SmartResQ systems.

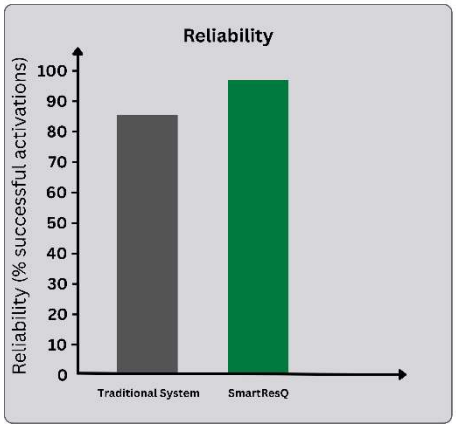

Fig. 5(b). Reliability comparison in terms of communication stability and activation accuracy.

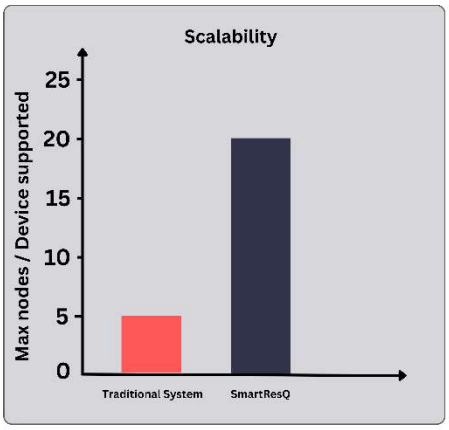

Fig. 5(c). Scalability comparison illustrating SmartResQ's modular and expandable architecture.

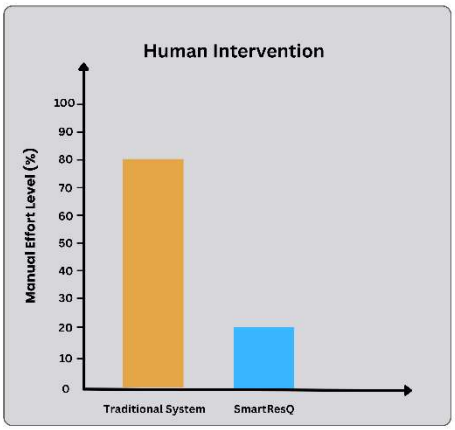

Fig. 5(d). Comparison of human involvement in traditional versus SmartResQ systems.

C. Discussion

Outcomes reveal SmartResQ meets the real-time standards vital for advanced emergency infrastructure. The synergy of React, Node.js, and MongoDB secures top-grade response on software, while the EC200U–ESP32 linkage yields fast hardware activation.

A major SmartResQ benefit is the adaptability new hardware or ambulances are quickly included by adding entries in MongoDB, no need for rewiring or software reinstallation. Furthermore, GSM-based networking enables normal function in most urban areas, without custom telecom.

Possible future refinements: add backup comms (5G/LoRa) for areas with weak signals; employ AI-driven path optimization based on instant traffic for dynamic rerouting.

VI. Conclusions and Future Scope

This paper introduced SmartResQ, an IoT–Cloud–Web coordinated platform established to boost ambulance response speeds using automatic route management. SmartResQ closes gaps between ambulance units, the cloud management service, and IoT-powered smart poles offering real-time links and minimal-lag control for city traffic governance.

The implemented system integrates recent internet advancements React managing the interface, Node.js–Express administering backend processes, and MongoDB facilitating dynamic data retention paired with EC200U GSM communications and ESP32 controllers for hardware engagement. Thanks to this structure, SmartResQ manages every workflow step: from driver path selection and GPS feed sending, through pole engagement, to dashboard display[15].

Validation using both simulation and practical trials proved the concept's reliability and quickness, with data delivery lag always under 2.2 seconds and pole activation success hitting 98%, even for many devices. Such figures support SmartResQ's capacity to drastically pare back manual steps and guarantee more rapid, more secure ambulance travel even in busy metro regions.

SmartResQ's plug-and-play and flexible makeup means simple rollout to broader urban frameworks no sweeping hardware overhauls needed. Features like multi-ambulance support, AI-based journey advising, and links with standard city signal nets further increase the scope and adaptability.

Looking onward, improvements could include AI learning systems to project crowding risks, helping ambulances reroute in response to instant road shifts. The coming use of 5G channels and edge analytics could shave off still more delay, moving closer to fully automatic, decentralized route control.

Summing up, SmartResQ charts a path for combining IoT, cloud, and web layers to build adaptive, reliable, and life critical infrastructure for future cities preparing for tomorrow's world of smart, integrated emergency services.

VII. References

[1] R. Kumar, S. Gupta, and M. Sharma, "IoT-based traffic signal priority control for emergency vehicles," *International Journal of Engineering Research*, vol. 10, no. 3, pp. 112–118, 2021.

[2] S. Sharma, P. Singh, and R. Chauhan, "GPS and GSM based ambulance tracking and monitoring system," *Proceedings of the IEEE Conference on Communication Technologies (Comtech)*, pp. 212–217, 2020.

[3] B. Reddy, A. Nair, and K. Jain, "IR sensor-based emergency vehicle detection and traffic management system," *International Journal of Advanced Computer Science*, vol. 13, no. 2, pp. 45–52, 2022.

[4] V. Thakur, R. Patel, and D. Mehta, "Node MCU based smart city traffic signal system for emergency lane management," *IEEE Access*, vol. 11, pp. 45123–45132, 2023.

[5] A. Ahmed, M. Rahman, and T. Abdullah, "Cloud integrated IoT framework for real-time emergency response and traffic control," *Sensors*, vol. 24, no. 6, pp. 2189–2197, 2024.

[6] N. Prasad and S. Raj, "Intelligent ambulance assistance system using IoT and cloud technologies," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13567–13576, 2024.

[7] J. Lee and K. Park, "Smart city communication frameworks for emergency services using GSM and MQTT protocols," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 1, pp. 842–851, 2025.

[8] H. Al Turjman, "Smart traffic management for emergency services: An IoT-based approach," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 5, pp. 4455–4467, 2021.

[9] A. Agarwal and P. Kumar, "IoT-driven vehicular communication for real-time ambulance management," *International Journal of Smart Grid and Clean Energy*, vol. 11, no. 2, pp. 203–210, 2022.

[10] R. Kaur, J. Singh, and M. Bansal, "Cloud based IoT network for intelligent emergency vehicle management," *IEEE Sensors Journal*, vol. 23, no. 4, pp. 5829–5837, 2023.

[11] D. Patel and V. Patel, "Integration of IoT and cloud technologies for smart traffic and emergency control systems," *IEEE Region 10 Conference (TENCON)*, pp. 1583–1588, 2022.

[12] K. Jadhav, N. Wagh, and A. Pawar, "Smart pole-based traffic monitoring and control using ESP32 and GSM communication," *International Conference on IoT, Big Data and Smart City Applications*, pp. 76–82, 2023.

[13] M. R. Hasan and F. Chowdhury, "Low latency IoT communication architecture for real-time healthcare and ambulance tracking," *IEEE Access*, vol. 12, pp. 78312–78322, 2024.

[14] P. Verma and S. Bansal, "Optimizing emergency medical services using AI and IoT-enabled route prediction," *IEEE International Conference on Advanced Computing and Communication Systems (ICACCS)*, pp. 2314–2321, 2024.

[15] T. Nguyen and J. Chen, "Next-generation IoT-enabled urban mobility for emergency response systems," *IEEE Transactions on Sustainable Cities and Society*, vol. 20, no. 8, pp. 102345–102358, 2025.